

Design of an Intrusion Detection and Prevention System Using Well-Known Open-Source Tools

Jhonatan Parada Torres

Dr. Merlinda Drini, Faculty Advisor

Queensborough Community College

Department of Engineering Technology

City University of New York Undergraduate Research Scholars

ABSTRACT

Intrusion Detection and Prevention Systems (IDPS) are a critical component of modern network security infrastructure. As cyberattacks grow in frequency and sophistication, organizations increasingly rely on automated tools to detect, alert on, and block malicious network activity. This research investigates the design and evaluation of an open-source IDPS stack using well-known, production-grade tools: Suricata, Snort3, and EveBox.

The primary experiment analyzed captured network traffic from a documented Lumma Stealer infection (2026-01-31 traffic analysis exercise). Both Suricata (with the Emerging Threats Open ruleset) and Snort3 (with Cisco Talos LightSPD rules) were used to replay and analyze the PCAP. Suricata achieved a Precision of 65%, Recall of 100%, and an F1 Score of 78%, while Snort3 achieved Precision of 33%, Recall of 100%, and an F1 Score of 50%. Both engines produced zero false negatives.

A secondary finding concerns infrastructure efficiency: deploying Suricata and EveBox via Docker containers consumed approximately 57% less memory than running the equivalent stack inside a QEMU/KVM virtual machine, saving roughly 1,620 MB of RAM. This work demonstrates that open-source IDPS tooling can reliably detect modern commodity malware when paired with regularly updated community rulesets.

ACKNOWLEDGMENTS

We would like to thank the CUNY Collaborative Research and Scholarship Program (CRSP) for providing the opportunity and support to conduct this undergraduate research. Gratitude is also extended to the open-source security community, including the developers of Suricata, Snort, and EveBox, as well as Proofpoint Emerging Threats for maintaining the ET Open ruleset, and `malware-traffic-analysis.net` for making publicly available the PCAP captures used in this study.

TABLE OF CONTENTS

ABSTRACT	iii
ACKNOWLEDGMENTS	iv
1 Introduction	1
2 Related Work	3
3 Problem Description	5
4 My Approach	6
4.1 Experimental Design	6
4.2 Traffic Replay	6
4.3 Alert Classification	7
4.4 Infrastructure Measurement	7
5 Environment Setup and Tooling	8
5.1 Host System	8
5.2 Suricata	8
5.3 Snort3	8
5.4 Lumma Stealer Traffic	9
6 Experimental Results	10
6.1 Suricata Results	10
6.2 Snort3 Results	13
6.3 Comparative Analysis	15

7	Performance and Infrastructure	17
7.1	Memory Overhead: VM vs. Docker	17
7.2	Suricata Rule Profiling	17
7.3	Alert Noise Reduction	18
8	Conclusion	19
	REFERENCES	21
A	Lumma Stealer Suricata Signature	24
B	Environment Configuration Notes	26

Chapter 1

Introduction

Network intrusion detection and prevention has become an essential discipline in cybersecurity as organizations face a growing volume of targeted attacks. According to industry reports, the average organization faces approximately 1,200 attempted intrusions monthly, with 41% of these being targeted intrusions that require automated prevention systems [4]. In 2024 alone, more than 1,600 confirmed data breaches were recorded globally, with over 85% attributed to targeted intrusion attempts.

An Intrusion Detection and Prevention System (IDPS) is a network security tool designed to monitor network traffic for suspicious activity, generate alerts upon detection, and, in prevention mode, actively block or drop malicious packets. Commercial IDPS products can be expensive and opaque; this research focuses on whether freely available, open-source alternatives can provide comparable detection effectiveness against real-world commodity malware.

This project was undertaken as part of the CUNY Collaborative Research and Scholarship Program (CRSP). The research environment consists of a Linux-based home lab running virtualized infrastructure, including a Debian virtual machine hosting Suricata and EveBox. A Snort3 deployment in a Docker container was also configured for comparative analysis. The principal threat examined is Lumma Stealer, an information-stealing malware that experienced a massive rise in detections during the second half of 2024 according to ESET telemetry [3].

The remainder of this paper is organized as follows: Chapter 2 reviews related work in the area of open-source IDPS; Chapter 3 formally describes the problem being addressed;

Chapter 4 presents the approach and methodology; Chapter 5 describes the environment setup and tooling; Chapter 6 presents experimental results; Chapter 7 discusses performance findings; and Chapter 8 concludes the paper.

Chapter 2

Related Work

Intrusion detection systems have been an active area of research since the late 1980s. Anderson’s 1980 technical report at SRI International first articulated the concept of monitoring audit trails for anomalous behavior [1], and Denning’s 1987 paper formalized the intrusion detection model [2]. Since then, IDPS research has bifurcated into two broad families: *signature-based* (or misuse-based) detection, which matches traffic against known attack patterns, and *anomaly-based* detection, which models normal behavior and flags deviations.

Snort, initially released by Martin Roesch in 1998, became the de facto standard for open-source signature-based IDS. The Snort rule language and the community of rule writers it spawned—including the Emerging Threats project (now maintained by Proofpoint)—established a shared vocabulary for describing network-layer attack indicators. Suricata, developed by the Open Information Security Foundation (OISF) and first released in 2010, extended Snort’s model with multi-threaded processing, native TLS/HTTPS inspection through JA3 fingerprinting, and a richer rule language including sticky buffers and transactional rule matching [5].

A recurring theme in the IDPS literature is the challenge of false positives. Systems that are too sensitive generate alert fatigue, causing analysts to miss genuine threats buried in noise. This tension is measured using **Precision** (the fraction of alerts that are true threats) and **Recall** (the fraction of actual threats that are detected). The **F1 Score**, the harmonic mean of Precision and Recall, provides a single balanced metric:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Recent work has examined the deployment of IDPS in containerized environments. The shift from bare-metal or virtual machine deployments to Docker containers has potential implications for resource efficiency, reproducibility, and ease of management. This research contributes an empirical data point on memory overhead differences between a KVM-based and Docker-based IDPS deployment.

The Lumma Stealer malware family, also known as LummaC2, is a commercially sold infostealer active since approximately 2022. Its C2 communication relies on TLS-encrypted connections to domains registered under unusual top-level domains such as `.su` (Soviet Union) and `.cc`. The use of JA3 TLS fingerprints and fast-flux DNS are hallmark behaviors of Lumma that make it a suitable test case for evaluating signature-based IDPS rules [3].

Chapter 3

Problem Description

The central problem addressed by this research can be stated as follows: given a network packet capture (PCAP) containing traffic produced by a real-world Lumma Stealer infection, can open-source IDPS tools reliably identify the malicious activity with acceptable levels of precision and recall?

More specifically, this research investigates three sub-problems:

1. Can Suricata with the Emerging Threats Open ruleset detect Lumma Stealer C2 communication in a PCAP replay scenario?
2. Can Snort3 with the Cisco Talos LightSPD ruleset detect the same activity?
3. How do the two engines compare in terms of standard detection metrics (TP, FP, FN, Precision, Recall, F1)?

A secondary problem concerns infrastructure: what is the memory cost of running a full IDPS stack (Suricata + EveBox) in a traditional KVM virtual machine versus a Docker container deployment? This is relevant for practitioners considering how to deploy IDPS tooling on resource-constrained hardware.

The Lumma Stealer infection scenario is well-defined: a single Windows host (10.1.21.58) is infected, communicates with multiple C2 domains over TLS, performs DNS lookups for those domains via the local domain controller (10.1.21.2), and conducts victim fingerprinting activity. Ground truth for classifying alerts as TP or FP was established by cross-referencing observed domains against known Lumma Stealer indicators of compromise (IOCs) from public threat intelligence sources.

Chapter 4

My Approach

4.1 Experimental Design

The approach taken in this research is empirical and comparative. A PCAP file captured during a documented Lumma Stealer infection ([malware-traffic-analysis.net](https://malware-traffic-analysis.net/2026-01-31/), 2026-01-31) was used as the ground-truth traffic corpus. This PCAP was replayed independently through Suricata and Snort3, and all generated alerts were manually classified as True Positives (TP) or False Positives (FP) based on knowledge of the Lumma Stealer infection scenario.

For Suricata, the ET Open ruleset was used (version dated 2026-03-23, 41.77 MB, containing `emerging-all.rules`). For Snort3, the Cisco Talos LightSPD ruleset was used in its maximum-detection policy mode. This choice deliberately compares the two most commonly deployed open-source rulesets for each engine, representing a realistic deployment scenario rather than a cherry-picked configuration.

4.2 Traffic Replay

Traffic was replayed using Tcpreplay with the `--topspeed` flag to replay packets as fast as possible. Without this flag, Tcpreplay replays at the original capture rate, which unnecessarily prolongs experiments [10]. Suricata was running as a native daemon on a Debian 13 virtual machine. Snort3 was run inside a Docker container using the `ciscotalos/snort3` image from Docker Hub [7].

4.3 Alert Classification

Each unique alert signature was manually reviewed. The classification criteria were as follows: an alert was classified as a **True Positive** if the source or destination IP was an infected host or a known Lumma Stealer C2 server, and the signature was semantically consistent with Lumma Stealer behavior (e.g., DNS lookups for `.su` or `.cc` C2 domains, TLS SNI matches to known malicious domains, JA3 fingerprint matches to the SoftEther profile). An alert was classified as a **False Positive** if the traffic was demonstrably benign (e.g., Windows connectivity probes, normal Active Directory LDAP/SMB traffic, Cloudflare CDN connections).

4.4 Infrastructure Measurement

Memory consumption was measured using the `free` command on the host Linux system (Linux 6.14.0-29-generic, Ubuntu 24.04). Baseline measurements were taken with all services stopped. Measurements were then taken with each deployment configuration active: (a) QEMU/KVM with Debian 13 VM running Suricata and EveBox, and (b) Docker containers running Suricata and EveBox via Evecctl. The difference in used memory between configurations was used to quantify the overhead of each deployment approach.

Chapter 5

Environment Setup and Tooling

5.1 Host System

All experiments were conducted on a personal laptop running Ubuntu 24.04 LTS with a 6.14.0-29-generic kernel. The system has 7,586 MB of total RAM and uses the IceWM desktop environment to minimize resource overhead.

5.2 Suricata

Suricata version 7.0.10 was installed on a Debian 13 (LXDE) virtual machine managed by QEMU/KVM via `virt-manager`. The network interface parameter in `suricata.yaml` was configured to use `enp1s0` (the correct interface for this VM), as the default `eth0` configuration caused rules to fail to trigger. The ET Open ruleset was managed using `suricata-update`. EveBox was deployed both as a Docker container (via `Evectl`) and as a native Debian package for alert visualization and management [9].

5.3 Snort3

Snort3 (version 3.x) was deployed using the official `ciscotalos/snort3` Docker image. The Talos LightSPD ruleset and the `runsnort.py` helper script were used to generate and execute the maximum-detection command. Note: the Emerging Threats Snort 2.9 ruleset was initially attempted but found to be incompatible with Snort3 due to rule syntax differences between Snort2 and Snort3 [11].

5.4 Lumma Stealer Traffic

The Lumma Stealer is an information-stealing malware distributed as a Malware-as-a-Service (MaaS). ESET telemetry recorded a massive rise in Lumma Stealer detections in the second half of 2024 [3]. The traffic used in this experiment was obtained from `malware-traffic-analysis.net` (exercise dated 2026-01-31) [8]. The scenario involved a single Windows host (10.1.21.58) infected with Lumma Stealer, communicating with C2 infrastructure over TLS.

Key Lumma Stealer indicators present in the traffic included:

- TLS SNI connections to `whitepepper.su` and `whooptm.cyou`
- DNS lookups for both C2 domains and the `communicationfirewall-security.cc` domain
- JA3 TLS fingerprints matching the SoftEther Windows client profile (a known Lumma Stealer fingerprint)
- Victim fingerprinting HTTP activity directed to the C2 host

Chapter 6

Experimental Results

6.1 Suricata Results

Suricata, using the ET Open ruleset, generated 20 unique alert types from the PCAP replay. Manual classification yielded **13 True Positives** and **7 False Positives**, with zero False Negatives. The resulting metrics were:

$$\text{Precision} = 65\% \quad \text{Recall} = 100\% \quad F_1 = 78\%$$

Table 6.1 presents the full Suricata alert breakdown.

Table 6.1: Suricata Alert Classification

#	Cnt	Signature	Src → Dst	Cls	Rationale
1	4	SURICATA STREAM ESTABLISHED packet out of window (SMB)	10.1.21.2 → 10.1.21.58	FP	TCP reassembly artifact from DC SMB traffic
2	4	SURICATA STREAM ESTABLISHED invalid ack (SMB)	10.1.21.58 → 10.1.21.2	FP	TCP stream anomaly on SMB; benign reassembly artifact
3	4	SURICATA STREAM Packet with invalid ack (SMB)	10.1.21.58 → 10.1.21.2	FP	Same as above; TCP artifact on normal SMB domain traffic

Continued on next page

#	Cnt	Signature	Src → Dst	Cls	Rationale
4	1	ET JA3 Hash – Possible SoftEther Client (communicationfirewall-secu:	10.1.21.58 → 104.21.9.36	TP	Lumma JA3 fingerprint; .cc domain confirmed C2
5	15	ET MALWARE Observed Win32/Lumma Stealer Related Domain (whitepepper.su) in TLS SNI	10.1.21.58 → 153.92.1.49	TP	Direct Lumma C2 domain hit in TLS SNI; high-confidence
6	13	ET JA3 Hash – Possible SoftEther Client (whitepepper.su)	10.1.21.58 → 153.92.1.49	TP	Lumma JA3 fingerprint to confirmed C2 host
7	1	ET DROP Spamhaus DROP Listed Traffic Inbound group 10	80.97.160.24 → 10.1.21.58	TP	Inbound from DROP-listed IP; corroborates active C2
8	2	ET DNS Query for .cc TLD (communicationfirewall-secu:	10.1.21.58 → 10.1.21.2	TP	Infected host resolving confirmed Lumma C2 domain
9	1	SURICATA STREAM ESTABLISHED packet out of window (TLS)	10.1.21.58 → 153.92.1.49	TP	Stream anomaly on confirmed C2 channel; malicious context
10	10	ET MALWARE Win32/Lumma Stealer CnC Domain in DNS Lookup (whitepepper.su)	10.1.21.58 → 10.1.21.2	TP	DNS lookup for confirmed Lumma C2 domain; direct match

Continued on next page

#	Cnt	Signature	Src → Dst	Cls	Rationale
11	10	ET DNS Query for .su TLD (whitepepper.su)	10.1.21.58 → 10.1.21.2	TP	.su lookup confirmed malicious (whitepepper.su = C2)
12	6	ET INFO HTTP Request to .su TLD (whitepepper.su)	10.1.21.58 → 153.92.1.49	TP	HTTP to confirmed C2 host; staging/finger- printing phase
13	1	ET MALWARE Lumma Stealer Victim Fingerprinting Activity (whitepepper.su)	10.1.21.58 → 153.92.1.49	TP	Primary alert; Lumma victim fingerprinting – highest confidence
14	2	ET MALWARE Win32/Lumma Stealer CnC Domain in DNS Lookup (whooptm.cyou)	10.1.21.58 → 10.1.21.2	TP	Second Lumma C2 domain resolved by infected host
15	1	ET MALWARE Observed Win32/Lumma Stealer Related Domain (whooptm.cyou) in TLS SNI	10.1.21.58 → 62.72.32.156	TP	TLS connection to second confirmed Lumma C2 domain
16	1	ET JA3 Hash – Possible SoftEther Client (whooptm.cyou)	10.1.21.58 → 62.72.32.156	TP	Lumma JA3 fingerprint on second C2 channel
17	1	SURICATA STREAM excessive retransmissions (TLS) – Cloudflare	104.21.46.67 → 10.1.21.58	FP	Cloudflare IP; TCP retransmission noise

Continued on next page

#	Cnt	Signature	Src → Dst	Cls	Rationale
18	1	SURICATA STREAM excessive retransmissions (TLS) – Cloudflare	10.1.21.58 → 104.17.25.14	FP	Cloudflare IP; TCP retransmission noise
19	2	ET JA3 Hash – Possible SoftEther Client (assets.adobedtm.com)	10.1.21.58 → 184.29.31.84	FP	Legitimate Adobe DTM CDN; JA3 match coincidental
20	1	ET INFO Microsoft Connection Test (www.msftconnecttest.com)	10.1.21.58 → 23.55.178.249	FP	Normal Windows connectivity probe; fully benign

The most significant True Positive alerts were the direct Lumma Stealer domain matches in TLS SNI (signatures 5 and 15), the DNS lookup detections for both C2 domains (signatures 10 and 14), and the victim fingerprinting alert (signature 13). The False Positives were predominantly TCP stream reassembly artifacts from normal Active Directory SMB traffic and Cloudflare CDN connections.

6.2 Snort3 Results

Snort3, using the Talos LightSPD maximum-detection policy, generated 9 unique alert types. Manual classification yielded **3 True Positives** and **6 False Positives**, with zero False Negatives. The resulting metrics were:

$$\text{Precision} = 33\% \quad \text{Recall} = 100\% \quad F_1 = 50\%$$

Table 6.2 presents the full Snort3 alert breakdown.

Table 6.2: Snort3 (Talos LightSPD) Alert Classification

#	Cnt	Signature	SID	Src → Dst	Cls	Rationale
1	7	INDICATOR-SCAN UPnP service discover attempt	1:1917:16	10.1.21.58 → 239.255.255.250:1900	FP	Standard Windows UPnP multicast; normal host behaviour
2	4	PROTOCOL-DNS dns response for rfc1918 10/8 address	1:13249:14	10.1.21.2 → 10.1.21.58	FP	DC responding with RFC1918 addresses; normal in AD
3	18	SERVER-OTHER Novell eDirectory LDAP buffer overflow attempt	1:44604:1	10.1.21.58 → 10.1.21.2:389	FP	Pattern-matches on Windows LDAP; Novell eDirectory not present
4	12	PROTOCOL-ICMP destination unreachable port unreachable	1:402:16	10.1.21.58 → 10.1.21.2	FP	Normal ICMP unreachable; network noise, not malicious
5	10	INDICATOR- COMPROMISE Suspicious .su dns query	1:27721:4	10.1.21.58 → 10.1.21.2:53	TP	DNS queries for whitepepper.su — confirmed Lumma C2

Continued on next page

#	Cnt	Signature	SID	Src → Dst	Cls	Rationale
6	4	SERVER-OTHER WolfSSL PSK extension buffer overflow attempt	1:59597:1	10.1.21.58 → external:443	FP	False match on legitimate TLS handshakes; WolfSSL not in use
7	4	INDICATOR- COMPROMISE Suspicious .cc dns query	1:28190:4	10.1.21.58 → 10.1.21.2:53	TP	DNS queries for communicationfirewall-security — confirmed C2
8	67	NETBIOS SMB samr named pipe creation attempt	1:38322:2	10.1.21.58 → 10.1.21.2:445	FP	Normal Windows domain authentication; expected in AD
9	1	MALWARE-CNC DNS Fast Flux attempt	1:57756:2	10.1.21.2 → 10.1.21.58	TP	DC returning fast-flux DNS response for Lumma C2 domain

Snort3’s True Positive alerts were limited to DNS-based detections: queries for the .su TLD, queries for the .cc TLD, and a DNS fast-flux detection. Notably absent were TLS SNI-based Lumma Stealer detections, which were the highest-confidence alerts in Suricata. This is attributable to the difference in ruleset scope: the ET Open ruleset includes Lumma Stealer-specific TLS SNI signatures, while the Talos LightSPD rules rely more heavily on generic behavioral indicators.

6.3 Comparative Analysis

Table 6.3 summarizes the comparative performance of both engines.

Table 6.3: Suricata vs. Snort3 Comparative Metrics

Metric	Suricata	Snort3 (LightSPD)	Notes
True Positives	13	3	Suricata detected all 3 Snort TPs plus 10 more
False Positives	7	6	Similar FP counts; different underlying rules
False Negatives	0	0	Neither engine missed a confirmed infection
Precision	65%	33%	Suricata 2× more precise with ET Open ruleset
Recall	100%	100%	Both achieved full recall on the PCAP
F1 Score	78%	50%	Suricata substantially outperforms on F1
Ruleset	ET Open (41.77 MB)	Talos LightSPD (max detection)	Rulesets differ significantly in scope
Deployment	Native on Debian VM	Docker container	Docker saves ~1,620 MB RAM vs. KVM VM

Both engines achieved perfect Recall (100%), meaning neither engine missed any of the confirmed Lumma Stealer infection indicators present in the traffic. However, Suricata’s Precision was nearly double that of Snort3 (65% vs. 33%), primarily due to the ET Open ruleset containing more precise, malware-family-specific signatures. The Snort3 FP rate was driven by generic protocol anomaly rules that fire on normal Windows Active Directory and CDN traffic.

Chapter 7

Performance and Infrastructure

7.1 Memory Overhead: VM vs. Docker

A secondary research question concerned the memory efficiency of different IDPS deployment strategies. Memory was measured using the `free` command at each configuration stage. Table 7.1 summarizes the results.

Table 7.1: Memory Usage by Deployment Configuration (MB)

Configuration	Total	Used	Free	Available
Baseline (all services stopped)	7,586	736	6,589	6,850
KVM daemon only (libvirtd)	7,586	769	6,536	6,817
KVM + Debian13 VM + Suricata	7,586	2,615	3,978	4,971
KVM + VM + Suricata + EveBox	7,586	2,832	3,637	4,754
Docker (Evectl) + Suricata + EveBox	7,586	1,212	5,228	6,374

The Docker-based deployment used 1,212 MB, compared to 2,832 MB for the KVM-based deployment — a savings of approximately 1,620 MB, or 57% less memory. This is a meaningful difference for practitioners deploying IDPS tooling on laptops or budget hardware where RAM is a limiting resource.

7.2 Suricata Rule Profiling

During this research, techniques for monitoring Suricata performance were investigated, including tools such as Zabbix integration and `suri-stats` for graphing alert statistics over

time. Rule profiling features in Suricata allow identification of computationally expensive rules, which is useful for tuning rulesets in high-throughput environments.

7.3 Alert Noise Reduction

In a real deployment, alert noise management is critical. This research used EveBox's `disable.conf` mechanism to suppress known-noisy, benign alerts such as `ET INFO Spotify`, which generates high alert volume from normal Spotify client traffic. This tuning step is representative of the ongoing operational work required to maintain an effective IDPS deployment.

Chapter 8

Conclusion

This research demonstrated that open-source IDPS tooling can reliably detect Lumma Stealer network activity in a PCAP replay scenario. Suricata with the ET Open ruleset achieved an F1 Score of 78% and perfect Recall, making it the more effective engine for this use case. Snort3 with Talos LightSPD achieved perfect Recall as well, but at a lower F1 Score of 50% due to a higher false positive rate from generic protocol anomaly rules.

A key practical finding is that *ruleset choice matters at least as much as engine choice*. Suricata’s advantage was largely attributable to the ET Open ruleset containing malware-family-specific TLS SNI and DNS signatures for Lumma Stealer, whereas the Talos LightSPD ruleset relied on more generic behavioral indicators. Future work should investigate whether Snort3 with the ET Snort3 ruleset achieves comparable performance to Suricata with ET Open.

The infrastructure comparison confirmed that Docker-based IDPS deployment is substantially more memory-efficient than KVM-based deployment, using 57% less RAM in this test. For resource-constrained environments, this argues in favor of containerized IDPS deployment.

Future work includes:

- Expanding the experiment to additional malware families and PCAP datasets.
- Investigating anomaly-based detection approaches as a complement to signature-based detection.
- Exploring automated rule tuning to reduce false positives without sacrificing recall.
- Evaluating Snort3 with the ET Open Snort3 ruleset to isolate ruleset from engine

effects.

REFERENCES

Bibliography

- [1] Anderson, J. P. (1980). *Computer Security Threat Monitoring and Surveillance*. Technical Report, James P. Anderson Co., Fort Washington, PA.
- [2] Denning, D. E. (1987). An Intrusion-Detection Model. *IEEE Transactions on Software Engineering*, 13(2), 222–232.
- [3] ESET. (2024). *ESET Threat Report H2 2024*. ESET Research.
- [4] Market Growth Reports. (2024). *Intrusion Detection System / Intrusion Prevention System (IDS/IPS) Market*. <https://www.marketgrowthreports.com/market-reports/intrusion-detection-system-intrusion-prevention-system-ids-ips-market-110928>
- [5] Open Information Security Foundation (OISF). (2024). *Suricata Documentation (v7.0)*. <https://docs.suricata.io>
- [6] Proofpoint Emerging Threats. (2026). *Emerging Threats Open Ruleset*. <https://rules.emergingthreats.net/>
- [7] Cisco Talos. (2024). *Snort3 Docker Image and LightSPD Rules*. <https://hub.docker.com/r/ciscotalos/snort3>
- [8] malware-traffic-analysis.net. (2026, January 31). *Traffic Analysis Exercise: Lumma in the Room-ah*. <https://www.malware-traffic-analysis.net/2026/01/31/index.html>
- [9] Jasonish. (2024). *EveBox: Web-based Event Viewer and Alert Management Tool*. <https://github.com/jasonish/evebox>
- [10] AppNeta. (2024). *Tcp replay: Suite of Utilities for Replaying Network Traffic*. <https://tcpreplay.appneta.com/>
- [11] Community Emerging Threats Forum. (2024). *Snort3, Snort2lua, and the*

Emerging Threats Snort 2.9 Ruleset. <https://community.emergingthreats.net/t/snort3-snort2lua-and-the-emerging-threats-snort-2-9-ruleset/475>

Appendix A

Lumma Stealer Suricata Signature

The following is the ET Open Suricata signature for detecting Lumma Stealer C2 traffic via TLS SNI. This rule was among the highest-confidence True Positives identified in the experiment.

```
alert tls $HOME_NET any -> any any (
  msg:"ET MALWARE Observed Win32/Lumma Stealer Related Domain
    (whitepepper.su) in TLS SNI";
  flow:established,to_server;
  tls.sni;
  bsize:14;
  content:"whitepepper.su";
  fast_pattern;
  nocase;
  reference:md5,dc518a45c58b82ed194c465ba1c73148;
  classtype:domain-c2;
  sid:2066542;
  rev:1;
  metadata:tls_state TLSEncrypt,
    created_at 2025_12_31,
    deployment Perimeter,
    malware_family Lumma_Stealer,
    confidence High,
    signature_severity Critical,
    updated_at 2025_12_31,
    mitre_tactic_id TA0011,
    mitre_tactic_name Command_And_Control,
    mitre_technique_id T1071,
```

```
mitre_technique_name Application_Layer_Protocol;  
target:src_ip;  
)
```

Listing A.1: ET Open Suricata rule for Lumma Stealer TLS SNI detection (SID 2066542)

Appendix B

Environment Configuration Notes

Suricata interface fix: The default Suricata quickstart guide uses `eth0`, but the Debian VM's interface is `enp1s0`. Updating the `af-packet` interface in `/etc/suricata/suricata.yaml` was required for rules to trigger correctly.

EveBox filter syntax: EveBox uses a Lucene-based query syntax for filtering alerts. The configuration file is located at `/etc/evebox/evebox.yaml`.

Docker memory measurement: All memory figures were captured using the `free` command on the Ubuntu 24.04 host. Values represent used memory after system caches are accounted for.

Tcpreplay note: The `--topspeed (-tK)` flag is required for efficient PCAP replay. Without it, replays run at the original capture rate, significantly extending experiment duration.

Snort3 maximum-detection command (auto-generated by `runsnort.py`):

```
snort -c lightspd/policies/3.1.0.0-0/maximum-detection.lua \
  --daq-dir /usr/local/lib/daq \
  -A alert_fast \
  --plugin-path lightspd/modules/3.9.0.0/ubuntu-x64/ \
  -q \
  -r /home/snorty/2026-01-31-traffic-analysis-exercise.pcap
```

Listing B.1: Snort3 command used for PCAP analysis